



UNIVERSIDADE
FEDERAL DO CEARÁ
CAMPUS QUIXADÁ

Gerência de Configuração

[Git] Desfazendo coisas

Profa. Lana Mesquita



Conteúdo

- `git commit --amend`
- `git restore`
- `git reset`
- `git revert`
- Dicas: ~ e ^
- Treino

Desfazer coisas no Git

Uma das grandes vantagens do uso do Git é a possibilidade de desfazer coisas e manter a integridade do conteúdo armazenado, assim como o histórico.

Porém é necessário ter **atenção**, pois nem tudo pode ser revertido depois de desfazer algo.

```
git commit --amend
```

git commit --amend

Acabou de executar um commit e percebeu que faltou algo?

O --amend permite alterar o último commit.

Lembre de antes de fazer o amend adicionar (*git add*) as alterações que precisam entrar nesse commit.

```
git commit --amend
```

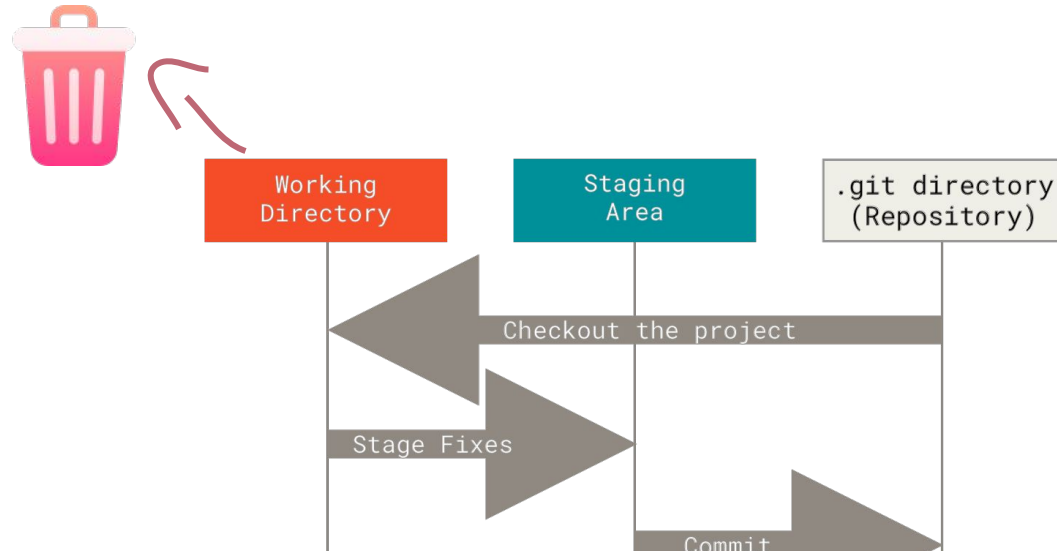
git restore

Jogando fora alterações que nem adicionou ainda

Atenção! Esse
comando não
tem volta.

```
git restore <arquivo>
```

Joga fora alterações não adicionadas.

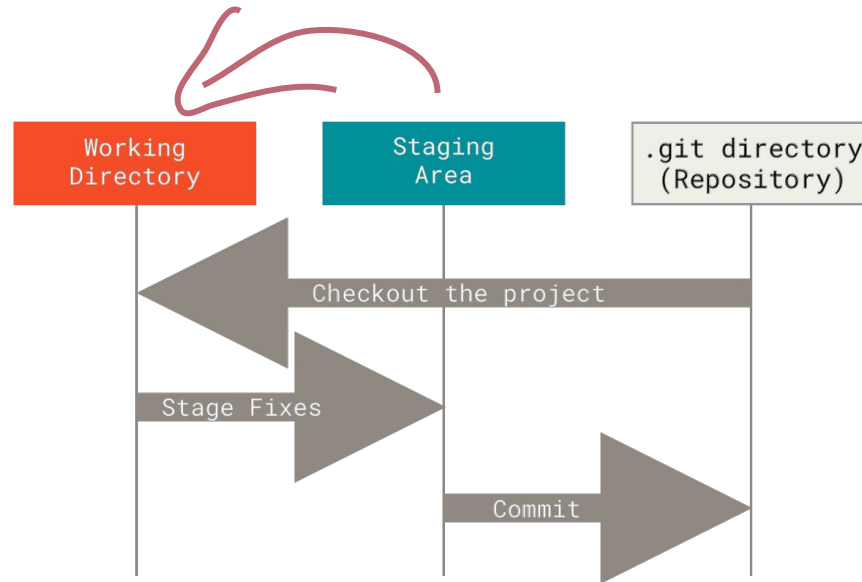


Retirando um arquivo do Stage

Também é possível
usar `git reset`
`<arquivo>` para a
mesma finalidade.

```
git restore --staged <arquivo>
```

O `<arquivo>` volta ao estado modificado, sem estar “pronto” (*staged*).



Voltando pra um commit específico

```
git restore --source=<ref> <file>
```

Exemplo:

```
git restore --source=HEAD~1 README.md
```

O arquivo README.md volta ao estado que tinha 1 commit antes do HEAD< isto é, 1 commit atrás.

Também é possível
usar `git reset`
`<arquivo>` para a
mesma finalidade.

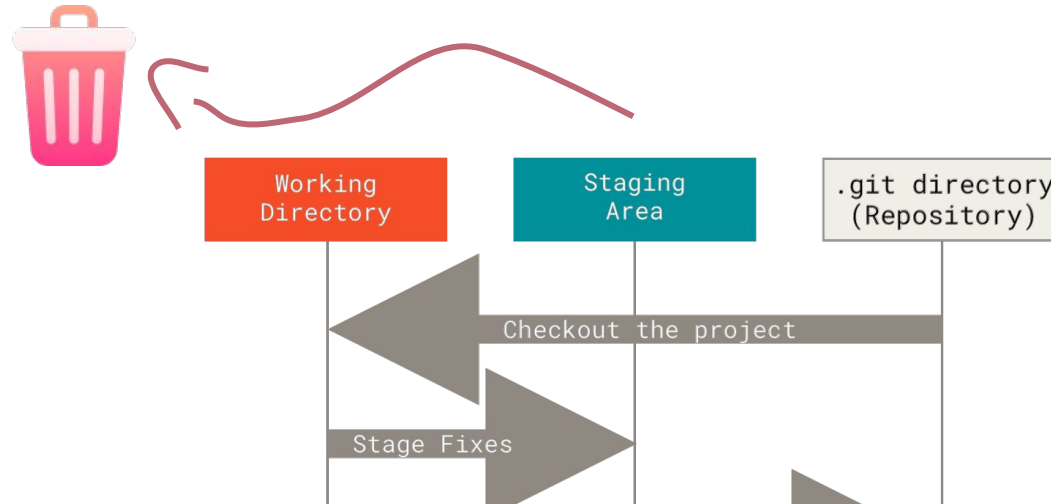
git reset

git reset --hard

o comando mais
temido do Git

```
git reset --hard
```

Realmente apaga suas últimas alterações, nem volta para o status modificado (*untracked* ou *modified*), a alteração só é apagada mesmo.



git revert

Reverter commits

A diferença desse comando é que ele mantém a deleção no histórico git, isto é, ele cria um commit que reverte outros commits.

```
git revert <commit-hash>
```

```
git revert <commit-hash> --no-edit
```

gera a mensagem de commit automaticamente.

```
git revert <commit-hash> --no-commit
```

gera as alterações sem fazer o commit, isto é, elas ficam como modificadas na área de trabalho.

Dicas: $\sim e^{\wedge}$

O acento circunflexo ^

^ (*caret*) se refere ao **primeiro pai de um commit**. Todo commit, exceto o inicial, tem pelo menos um pai. Em um fluxo de trabalho linear (onde não há merges), HEAD^ se refere ao commit imediatamente anterior ao HEAD.

O uso mais comum é com HEAD^, mas você pode usar com qualquer commit. Por exemplo, main^ se refere ao pai do último commit do ramo main.

Exemplo:

```
git restore --source=HEAD^ config.js
```

O acento circunflexo ^

Se um commit tem múltiplos pais (como em um **merge**),
pode especificar qual pai quer referenciar.

Por exemplo, **HEAD^2** se refere ao segundo pai do
commit atual.

0 til ~

~ (*tilde*) se refere a um ancestral direto, ou seja, um commit que está há um certo número de gerações atrás na mesma linha de desenvolvimento.

Por exemplo, **HEAD~3** ou **HEAD~~~** se refere ao commit que está três commits antes do HEAD (o commit atual).

```
git reset --hard HEAD~
```

A cardinal ^ + o til ~

É possível combinar referências ^ e ~ para criar navegações mais complexas.

A ordem é importante e a leitura deve ser feita da direita para a esquerda.

Por exemplo, **HEAD~2^3** significa:

1. Comece no HEAD.
2. Vá dois commits para trás na linha de desenvolvimento (HEAD~2).
3. Deste novo ponto, encontre o terceiro pai (^3).

Hash incompleto

No Git pode usar apenas uma parte do hash, como

```
git checkout 9e32a2
```

e ele tentará encontrar a referência.

Às vezes basta poucos dígitos.

Treino

Vamos testar!

[https://learngitbranching.js.org/?locale=pt BR&NODEMO=](https://learngitbranching.js.org/?locale=pt_BR&NODEMO=)



Dúvidas?
Manda no Discord!



Até a próxima aula!

Lana Mesquita

